Studio Nx

JS 304 — 장치 화면의 Sampler 연결

2026-10-21

JavaScript 304 — 장치 화면의 Sampler 연결 (`pages/sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`, `studionx/frontend/static/css/pages/studio.css`
작업	S1-11c Sampler MIDI 연주: 카드 부제와 경고, Mode 버튼 줄, Sample 줄 연결, One-Shot에서 무시되는 칸 표시 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 303 샘플 고르기, 다음 없음 (S1-11c의 마지막 문서)

1. 하는 일

악기 카드와 상세 창은 `param_schema`로 자동 생성됩니다(S1-09). 그런데 Sampler에는 노브, 셀렉트, 토글로는 표현할 수 없는 칸이 둘 생겼습니다. 샘플을 고르는 칸과, 목업처럼 이어 붙은 버튼으로 보여 주는 모드 칸입니다. 이 문서는 자동 생성 UI에 그 둘을 더한 부분을 설명합니다.

스키마의 새 항목	그려지는 모양
`type: "sample"` (`sample_id`)	303의 Sample 줄
`type: "select"` + `display: "segment"` (`mode`)	`Classic`, `One-Shot` 버튼 줄과, 고른 옵션의 `hint` 설명
`unit: "note"` (`root_note`)	노브 값을 숫자(60) 대신 노트 이름(C4)으로 표시

스키마에 필드를 더한 것이므로 다른 플러그인이 `display: "segment"`나 `hint`를 쓰면 같은 모양으로 그려집니다.

2. 언제 쓰나

쓰는 곳	하는 일
악기 칸을 그릴 때 `cardHtml`	Sampler 카드에 부제와 경고를 붙입니다.
카드를 눌러 상세 창을 열 때 `renderModalParams`	칸을 그리고 동작을 연결한 뒤, One-Shot에서 무시되는 칸을 흐리게 합니다.
시험 `ui_sampler.mjs`	카드, 상세 창, 저장, 소리를 실제 화면에서 확인합니다.

3. 코드

3.1 카드

```
export function cardHtml(kind, track, d) {
  const p = pluginById(d.plugin_id);
  const name = p ? p.name : 'Plugin #' + d.plugin_id;
  const values = withDefaults(p, d.params);
  const prim = primaryItems(p);
  const bypassed = kind === 'EFFECT' && d.is_bypassed;
  const idAttr = kind === 'EFFECT'
```

```
    ? ' data-track-plugin-id="' + d.track_plugin_id + '"'
    : ' data-track-instrument-id="' + d.track_instrument_id + '"';
  const knobs = prim.map(function (item) {
    return '<span class="sq-knob" role="slider" tabindex="0" data-param-key="' + esc(item.key) + '"'
      + ' aria-label="' + esc(item.name) + '" aria-valuemin="' + numOr(item.min, 0) + '"'
      + ' aria-valuemax="' + numOr(item.max, 1) + '" aria-valuenow="' + numOr(values[item.key], 0) +
'">'
      + '<i style="--a:' + knobAngle(item, values[item.key]) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, values[item.key])) + '</span>'
      + '<span class="sq-knob__name">' + esc(item.name) + '</span></span>';
  }).join('');
  const mix = kind === 'INSTRUMENT' ? instrumentMixHtml(d) : effectExtraHtml();
  // Sampler 는 이름 아래에 "샘플 이름 · 모드" 를, 샘플이 없으면 경고를 보여 줌 (S1-11c)
  let sub = '';
  let noSample = false;
  if (kind === 'INSTRUMENT' && isSampler(p)) {
    const info = cardSubtitle(sampleInfoOf(values.sample_id), values.sample_id, selectLabel(schemaItem
(p, 'mode'), values.mode));
    noSample = info.empty;
    sub = '<span class="sq-fx__sub' + (info.empty ? ' is-warn' : '') + '">' + (info.empty ? '⚠ ' : '') +
esc(info.text) + '</span>';
  }
  return '<li class="sq-fx sq-fx--dev' + (bypassed ? ' is-bypassed' : '') + (noSample ? ' is-nosample' :
'') + '" draggable="' + String(kind === 'EFFECT') + '"'
    + ' data-device-kind="' + kind + '"' + idAttr + '>'
    + '<span class="sq-fx__art" aria-hidden="true">' + artHtml(p) + '</span>'
    + '<div class="sq-fx__head"><span class="sq-fx__grip" aria-hidden="true">' + (kind === 'EFFECT' ?
'⠿' : '⏻') + '</span>'
    + '<span class="sq-fx__name">' + esc(name) + '</span>'
    + (kind === 'EFFECT'
      ? '<button type="button" class="sq-fx__power" aria-pressed="' + String(!d.is_bypassed) + '" aria-l
abel="Bypass">⏻</button>'
      : '')
    + '<button type="button" class="sq-fx__del" aria-label="Remove ' + (kind === 'EFFECT' ? 'effect' :
'instrument') + '">✕</button></div>'
    + sub
    + '<div class="sq-knobs">' + knobs + '</div>'
    + '<span class="sq-fx__preset">Preset: ' + esc(d.__preset || 'Init') + '</span>'
    + mix + '</li>';
}
```

3.2 상세 창의 칸

```
export function renderModalParams() {
  const box = document.getElementById('device-params');
  if (!box || !modal) return;
  const schema = (modal.plugin.param_schema || []).filter(function (item) {
    return (item.group || 'General') === modal.group;
  });
  box.innerHTML = schema.map(function (item) { return paramCellHtml(item); }).join('')
    || '<p class="sq-cell__hint">This plugin has no parameters.</p>';
  schema.forEach(function (item) {
    const cell = box.querySelector('[data-param-cell="' + item.key + '"]');
    if (cell) bindModalParam(cell, item);
  });
  // One-Shot 모드에서 무시되는 칸은 흐리게 (S1-11c)
```

```
  const ignored = isSampler(modal.plugin) ? ignoredKeys(modal.draft.mode) : [];
  ignored.forEach(function (key) {
    const cell = box.querySelector('[data-param-cell="' + key + '"]');
    if (cell) {
      cell.classList.add('is-ignored');
      cell.title = 'Ignored in One-Shot mode';
    }
  });
}
```

```
function segmentHtml(item, v) {
  const opts = item.options || [];
  const cur = opts.filter(function (o) { return String(o.value) === String(v); })[0];
  const buttons = opts.map(function (o) {
    const on = String(o.value) === String(v);
    return '<button type="button" data-seg="' + esc(o.value) + '" class="' + (on ? 'is-on' : '') + '" ar
ia-pressed="' + String(on) + '">' + esc(o.label) + '</button>';
  }).join('');
  return '<div class="sq-param sq-param--wide sq-segcell" data-param-cell="' + esc(item.key) + '">'
    + '<span class="sq-samp__label">' + esc(item.name) + '</span>'
    + '<div class="ex-seg" role="group" aria-label="' + esc(item.name) + '">' + buttons + '</div>'
    + (cur && cur.hint ? '<p class="sq-segcell__hint">' + esc(cur.hint) + '</p>' : '')
    + '</div>';
}
```

```
function paramCellHtml(item) {
  const v = modal.draft[item.key];
  if (item.type === 'sample') return samplePickHtml(item, v, modal.draft.root_note);
  if (item.type === 'select' && item.display === 'segment') return segmentHtml(item, v);
  let control;
  if (item.type === 'select') {
    const opts = (item.options || []).map(function (o) {
      return '<option value="' + esc(o.value) + '"' + (String(o.value) === String(v) ? ' selected' : '')
+ '>' + esc(o.label) + '</option>';
    }).join('');
    control = '<select data-modal-param="' + esc(item.key) + '" aria-label="' + esc(item.name) + '">' +
opts + '</select>';
  } else if (item.type === 'toggle') {
    control = '<span class="sq-switch" role="switch" tabindex="0" data-modal-param="' + esc(item.key) +
'" aria-label="' + esc(item.name) + '" aria-pressed="' + String(!!v) + '"></span>';
  } else {
    control = '<span class="sq-knob" role="slider" tabindex="0" data-modal-param="' + esc(item.key) + '"
aria-label="' + esc(item.name) + '"'
      + ' aria-valuemin="' + numOr(item.min, 0) + '" aria-valuemax="' + numOr(item.max, 1) + '" aria-val
uenow="' + numOr(v, 0) + '">'
      + '<i style="--a:' + knobAngle(item, v) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, v)) + '</span></span>';
  }
  return '<div class="sq-param" data-param-cell="' + esc(item.key) + '">' + control
    + '<span class="sq-param__name" title="' + esc(item.key) + '">' + esc(item.name) + '</span></div>';
}
```

```
function bindModalParam(cell, item) {
  if (item.type === 'sample') {
    bindSamplePick(cell, modal.draft[item.key], function (id) {
      setDraft(item.key, id);
```

```
      renderModalParams();
    });
    return;
  }
  if (item.type === 'select' && item.display === 'segment') {
    Array.prototype.forEach.call(cell.querySelectorAll('[data-seg]'), function (b) {
      b.addEventListener('click', function () {
        setDraft(item.key, b.getAttribute('data-seg'));
        renderModalParams();
      });
    });
    return;
  }
  const el = cell.querySelector('[data-modal-param]');
  if (!el) return;
  if (item.type === 'select') {
    el.addEventListener('change', function () { setDraft(item.key, cell.querySelector('select').value);
});
    return;
  }
  if (item.type === 'toggle') {
    // 토글은 끌지 않고 누를 때마다 켜짐 · 꺼짐만 바뀝니다
    const flip = function () {
      const next = el.getAttribute('aria-pressed') !== 'true';
      el.setAttribute('aria-pressed', String(next));
      setDraft(item.key, next);
    };
    el.addEventListener('click', flip);
    el.addEventListener('keydown', function (e) {
      if (e.key === ' ' || e.key === 'Enter') {
        e.preventDefault();
        flip();
      }
    });
    return;
  }
  bindControl(el, {
    item: item,
    value: numOr(v0(modal.draft, item.key), 0),
    render: function (v) { paintKnob(el, item, v); },
    commit: function (v) { setDraft(item.key, v); },
  });
}
```

3.3 값 표시

```
export function fmtValue(item, v) {
  if (!item) return String(v);
  if (item.type === 'toggle') return v ? 'On' : 'Off';
  if (item.type === 'select') return selectLabel(item, v);
  if (item.unit === 'note') return noteName(v);   // Sampler 의 Root 는 노트 이름(C4)으로
  const step = item.step ? item.step : 0;
  const dec = step >= 1 ? 0 : (step >= 0.1 ? 1 : (step >= 0.01 ? 2 : 3));
  const unit = item.unit ? (item.unit === '%' ? ' %' : ' ' + item.unit) : '';
```

```
  return Number(v).toFixed(dec) + unit;
}
```

4. 동작 순서

```
Mode 버튼 클릭 ─▶ 초안(draft.mode)을 바꾸고 renderModalParams 로 칸을 다시 그림
                  ├─ segmentHtml 이 고른 옵션의 hint 를 다시 보여 줌
                  └─ ignoredKeys('oneshot') = loop, sustain, release 칸에 is-ignored (흐리게, 설명 말풍선)

샘플 선택     ─▶ 초안(draft.sample_id)을 바꾸고 칸을 다시 그림 ─▶ Save ─▶ PATCH params 에 sample_id, mode 가 함께 저장
Save 뒤       ─▶ refreshCells 가 카드를 다시 그려 부제가 "샘플 이름, 모드"로 바뀜
```

카드의 모양	조건
부제 `kick_808, Classic`	샘플이 붙어 있음
경고 `⚠ No sample chosen. Notes make no sound.`와 점선 테두리	`sample_id`가 없음
부제 `Sample #12, Classic`	샘플 번호는 있으나 목록에서 이름을 아직 못 찾음 (목록 읽기 전이거나 실패)

카드의 핵심 노브는 `type: "knob"`이고 `primary: true`인 항목만 모으므로, Sample과 Mode는 노브로 나타나지 않고 Gain, Tune, Decay가 올라옵니다.

5. 최신 문법으로 쓰면

```
// segmentHtml 을 최신 문법으로 쓴 모양
const segmentHtml = (item, v) => {
  const cur = (item.options ?? []).find((o) => String(o.value) === String(v));
  const buttons = (item.options ?? []).map((o) => {
    const on = String(o.value) === String(v);
    return `<button type="button" data-seg="${esc(o.value)}" class="${on ? 'is-on' : ''}" aria-pressed="${on}">${esc(o.label)}</button>`;
  }).join('');
  return `<div class="sq-param sq-param--wide sq-segcell" data-param-cell="${esc(item.key)}">…${buttons}…${cur?.hint ? `<p>${esc(cur.hint)}</p>` : ''}</div>`;
};

// One-Shot 에서 무시되는 칸 표시
const ignored = isSampler(modal.plugin) ? ignoredKeys(modal.draft.mode) : [];
ignored.forEach((key) => box.querySelector(`[data-param-cell="${key}"]`)?.classList.add('is-ignored'));
```

전통 문법 (실제 코드)	최신 문법
`filter(...)[0]`	`find(...)`
`'<button ...' + x + '...'`	템플릿 리터럴
`if (cell) { cell.classList.add(...) }`	옵셔널 체이닝 `cell?.classList.add(...)`
`cur && cur.hint ? ... : ''`	`cur?.hint ? ... : ''`

6. 왜 이렇게

선택	이유
새 칸 종류를 스키마의 `type`과 `display`로 표현했습니다	플러그인 이름(`sampler`)을 화면 코드에 박지 않고, 스키마가 모양을 정하게 하는 기존 방식(S1-09 자동 생성)을 지켰습니다. 나중에 Drum Rack이나 다른 악기가 샘플 칸을 쓰면 코드를 고치지 않아도 됩니다.
무시되는 칸 목록(`ignoredKeys`)만 Sampler 전용입니다	"One-Shot에서 Loop, Sustain, Release를 무시한다"는 규칙은 Sampler의 동작에 속하므로 화면의 일반 규칙으로 만들지 않았습니다. 목록이 순수 함수라 시험하기 쉽습니다.
무시되는 칸을 숨기지 않고 흐리게 합니다	칸이 사라지면 모드를 바꿨을 때 노브 배치가 흔들립니다. 흐리게 두고 값은 보존하면 Classic으로 돌아갔을 때 설정이 그대로 있습니다. 목업(`02_oneshot.png`)도 같은 모양입니다.
모드를 바꾸면 칸 전체를 다시 그립니다	힌트 문구와 흐림 표시가 함께 바뀌어야 하고, 초안은 `modal.draft`에 있어서 다시 그려도 값이 사라지지 않습니다. 노브를 만질 때는 다시 그리지 않아 드래그가 끊기지 않습니다.
다른 DB 변경은 없습니다	값은 기존 `track_instruments.params`(JSON)에 `sample_id`와 `mode`로 들어갑니다. 이미 붙어 있던 Sampler는 키가 빠져 있어도 `withDefaults`와 `resolveParams`가 기본값(샘플 없음, `classic`)으로 채웁니다.
카드의 부제는 `values`(기본값을 깐 params)로 만듭니다	저장된 params에 `mode`가 아직 없는 옛 Sampler도 "Classic"으로 표시되어야 합니다.

7. 시험

- `docs/handoff/s1-11c_tests/ui_sampler.mjs` (39건)가 이 문서의 모든 동작을 실제 화면에서 확인합니다.
  - 상세 창은 Sampler로 열리고 탭이 Sample, Amp Env입니다.
  - Mode 줄은 Classic이 눌린 채로 시작하고 Classic 설명이 보입니다. One-Shot을 누르면 설명이 바뀌고, Loop(Sample 탭)와 Sustain, Release(Amp Env 탭)가 흐려지며, Attack은 흐려지지 않습니다. 고치면 "Unsaved changes"가 됩니다.
  - 저장하면 서버의 `params`에 `sample_id`와 `mode`가 들어가고, 카드 부제가 바뀝니다.
  - 샘플을 고른 뒤에도 Mode 선택이 유지됩니다.
- 스크린샷 7장은 `docs/screenshots/2026-10-21_s1-11c_sampler/`에 있습니다.
- 화면 자체의 문제는 시험에서 나오지 않았습니다. 첫 실행의 실패 4건은 시험 기대값의 오류였고(303 참고), 실제 화면은 목업과 같은 모양으로 나왔습니다.